Annexe 17.9 - Charte GitLab, modèle de MR et `README.md` du dépôt

1. Ouverture et accès (semaine 1)

1. Créer le dépôt **privé** `minishop-sae-<groupe>` sur le GitLab de l'établissement (ou `gitlab.com` si imposé).

2. Ajouter les 3 développeurs (**Developer**) et l'enseignante (**Reporter**, au minimum, pour pouvoir lire le code, les MR et l'historique).

3. Envoyer le courriel obligatoire : à `thanh-phuong.nguyen@univ-cotedazur.fr`, **sujet `Composition SAE`**, corps : noms/prénoms/numéros étudiants, rôle de chacun (cf. RACI §3.3 du CDC), **URL du dépôt**.

4. Protéger `main` (pas de push direct) ; activer les MR ; **désactiver le squash** pour conserver les commits individuels ; activer « Delete source branch ».

5. Tag initial `v0-cadrage` ; tags de jalons `v1-mcd`, `v2-base`, `v3-front`, `v4-back`, `v5-rendu`.

2. Découpage des branches (une branche = une MR = un lot du §13.3)

```
feat/L04-mcd-mld            docs/L02-cahier-des-charges     feat/L06-procedures-stockees
feat/L05-script-sql         feat/L07-triggers               test/L14-suite-sql
feat/L08-socle-mvc          feat/L09-catalogue-visiteur     feat/L10-panier-commande
feat/L11-back-office        feat/L12-securite-csrf          feat/L13-js-panier
fix/T09-rollback-stock      docs/L16-support-soutenance
```

3. Modèle de description de MR (à coller dans le gabarit `.gitlab/merge_request_templates/Lot.md`)

```
## Lot concerné
L10 - panier + commande

## Exigences couvertes
EF-CLI-03, EF-CLI-04, EF-CLI-05, EF-CLI-06, EF-CLI-07
Règles touchées : RB-04, RB-05, RB-06, RB-15, RB-18

## Ce que ça change (côté base / PHP / IHM)
- `sp_add_order_line` : verrou FOR UPDATE + agrégation des quantités déjà réservées
- `CommandeRepository::confirmerDepuisPanier()` : un seul CALL, transaction, traduction des codes
- `public/js/panier.js` : mises à jour par textContent (aucun innerHTML)

## Preuves
- [x] `scripts/run_sql_tests.sh` -> 29/29
- [x] `php -l` sur les fichiers modifiés
- [x] capture du parcours F-17 (commande créée, stock 16 -> 14)
- [x] `grep` CI n° 1, 2, 5 : aucun résultat
```

```
## Régression vérifiée
commande multi-lignes, annulation puis re-commande, panier hors session.

## Auteur / relecteur
Auteur : @dev-c - Relecteur : @dev-b (obligatoire, autre que l'auteur)
```

4. Convention de messages de commit

```
<type>(<périmètre>): <impératif, minuscule, <= 72 caractères>

type ∈ { feat, fix, refactor, test, docs, style, chore, security }
périmètre ∈ { base, sql, modele, repo, controller, vue, js, secu, tests, docs, ci, readme }

exemples :
feat(sql): ajoute sp_create_order_from_basket avec transaction unique
fix(vue): échappe le nom du produit dans la fiche (e() manquant)
test(base): T-11 vérifie le rejet et non la réparation du prix d'une ligne
security(session): régénère l'id de session à l'ouverture et à la montée de rôle
```

5. Ce qui ne doit **jamais** être commité

`app/Config/env.php` (identifiants MySQL), tout fichier `.env` réel, les journaux `var/log/*`, les dumps SQL contenant des données réelles, `vendor/`, `node_modules/`, les captures d'écran de test (référencées par un lien wiki ou compressées dans `tests/captures/` si légères), les hashes de mots de passe autres que ceux de démonstration.

6. Modèle de `README.md` livré (exactement ce fichier à la racine du dépôt)

Le fichier `README.md` du dépôt (reproduit intégralement dans le dépôt de ce rendu) contient : présentation du projet, membres et rôles, prérequis, installation en 5 commandes, comptes de démonstration avec **mots de passe de démo**, exécution des tests (SQL, charge, statiques), arborescence, lien vers les documents (CDC, tests, soutenance), procédure de sauvegarde/restauration, et la table des 29 tests SQL avec leur commande de rejeu. **Critère d'acceptation** : un tiers, sur une machine propre, doit pouvoir installer et lancer la démo en <= 15 min en suivant uniquement le README (`ENF-13`).

7. CI : le fichier de contrôle livré (`/home/user/.gitlab-ci.yml`, 6 jobs)

```
# .gitlab-ci.yml (extrait des jobs obligatoires)
stages: [validate, db, test, docs]
php-lint:      { stage: validate, script: [ "find app public -name '*.php' -exec php -l {} \\;" ]
}
security-scan: { stage: validate, script: [ "bash tests/security/controles.sh" ] }   # 10 grep
```

```
bloquants
base-tests:
  stage: db
  services: [ mysql:8.0 ]
  variables: { MYSQL_DATABASE: minishop_test, MYSQL_ALLOW_EMPTY_PASSWORD: "yes", DB_TEST:
minishop_test }
  before_script: [ "apt-get update -qq && apt-get install -y -qq default-mysql-client" ]
  script: [ "DB_USER=root bash scripts/run_sql_tests.sh" ]
  artifacts: { paths: [ "tests/sql/rapport_tests_sql.md", "tests/sql/out" ] }
```

Règle « pas de re-run » (à appliquer telle quelle). Si le pipeline est **vert avant le merge** (statut de la MR : « Pipeline passed »), **aucun re-run n'est nécessaire après le merge** : le pipeline post-merge rejoue les 6 mêmes jobs, sur le même arbre de fichiers (GitLab CI teste la MR sur son merge result, GitHub Actions sur `refs/pull/<n>/merge`). Le run post-merge sert de **preuve et de badge**, pas de contrôle - on ne clique donc ni « Retry » côté GitLab, ni « Re-run jobs » côté GitHub. Exceptions, à connaître pour la soutenance : `main` a avancé entre le run de la MR et le merge (la chaîne re-démarre seule), run **annulé** (`interruptible: true`), échec de code corrigé puis poussé (c'est le `push` qui relance, jamais un re-run manuel), `style` en échec `allow_failure` (non bloquant), **panne d'infrastructure** (un seul Retry justifié, anomalie consignée). Tableau complet : **CDC §14.2**.

8. Ce que le jury regarde en premier (auto-contrôle avant le rendu)

Point	Où le prouver
Pipeline **vert avant le merge** (et non un re-run après coup)	onglet *Pipelines* de la MR « Pipeline passed » (GitHub : `gh pr checks <n>`) ; **règle** « pas de re-run » du §7
Les 6 jobs tournent sans `allow_failure` déguisé	seul `style` est en `allow_failure: true` (§7)
Pipeline qui **casse** sur une concaténation SQL	contrôle `security-scan` n° 1 (`tests/security/controles.sh`)